

MOG2 + YOLO Pipeline

Stress Test Findings & Pipeline Analysis

Rail Grade Crossing Monitoring and Alert System

1. Stress Test Video Overview

The stress test video used for this analysis is real surveillance footage captured by a fixed overhead camera at an active railroad grade crossing. The footage was sourced from a FOX13 news broadcast and documents an actual near-miss incident involving a white SUV trapped on the tracks.

Property	Value
Resolution	1772 x 872
Frame Rate	31.44 FPS
Total Frames	1,027
Duration	~32.7 seconds
Camera Type	Fixed overhead surveillance camera

1.1 Incident Timeline

The following key events occur throughout the video and were used to select snapshot timestamps for pipeline evaluation:

Time	Event
0s	Gates already lowering — SUV entering crossing
4s	White SUV fully stopped on the tracks
7s	Gates fully down — SUV trapped inside crossing
10s	SUV attempts to reverse, strikes gate arm
14s	SUV reverses further — gate arm visibly bent/damaged
23s	Driver exits the SUV
28s	Train enters the frame approaching the crossing
32s	Video ends just before train impact

2. Pipeline Configuration for This Video

The following parameters were updated from the original test video to match the new footage:

2.1 ZOI Points (Cell 3)

Zone of Interest coordinates were re-tuned to cover the right-side road crossing where the incident occurs. The left-side tracks were excluded as they are not involved in the incident.

```
# Coordinate system: X increases right, Y increases down
zoi_points = np.array([
    [x1, y1], # TL
    [x2, y2], # TR
    [x3, y3], # BR
    [x4, y4], # BL
], dtype=np.int32)
```

2.2 Capture Timestamps (Cells 7 and 12)

Snapshot timestamps were updated to reflect meaningful moments in the new video. Timestamps are calculated dynamically using FPS rather than hardcoded frame numbers to ensure compatibility across videos of different lengths.

```
capture_times = {
    "0s - SUV entering as gates come down" : 0,
    "7s - SUV fully trapped on tracks"      : 7,
    "14s - Gate damaged, SUV still on tracks": 14,
    "28s - Train entering frame"           : 28
}
```

2.3 MOG2 Test Frame (Cell 8)

The test frame in Cell 8 was changed from a hardcoded frame number to a dynamic calculation using seconds multiplied by FPS, with a safety clamp to prevent out-of-bounds access on shorter videos:

```
test_time = 14 # seconds
test_frame_num = min(int(test_time * fps), total_frames - 1)
```

3. ZOI Stabilization Results (Cells 4–7)

The Lucas-Kanade optical flow and RANSAC homography pipeline performed reliably throughout the video. The fixed camera angle resulted in minimal camera shake, keeping the ZOI polygon well-anchored to the crossing across all frames.

Timestamp	Frame	Inliers	Observation
0s	1	180	Empty crossing — full inlier count
7s	216	152	SUV on tracks — moving objects reduce inliers
14s	437	152	Gate damage visible — inlier count stable
28s	877	147	Train approaching — slight inlier reduction

The gradual reduction in inlier count from 180 to 147 is expected as more moving objects (SUV, damaged gate, approaching train) enter the frame and are correctly classified as outliers by RANSAC. A count above 100 is considered reliable for stable ZOI positioning.

4. MOG2 + YOLO Detection Results (Cells 8–11)

4.1 MOG2 Background Learning

MOG2 was trained on the first 300 frames (~9.5 seconds) of the video. Because the incident begins at approximately 1 second, the SUV is present during part of the training window. Despite this, MOG2 successfully learned the background and flagged anomalies in later frames. This represents a known limitation — ideally, MOG2 would train on a fully empty crossing.

4.2 YOLO Object Detection

YOLO (YOLOv11n) was applied to the test frame at 14 seconds. The following detections were made:

Object	Confidence	Notes
Truck	0.60	White SUV classified as truck due to camera angle and vehicle size
Car	0.35	Partial vehicle visible at right frame edge — low confidence expected

The misclassification of the SUV as a truck is not a problem for the pipeline. What matters is that YOLO identifies it as a known vehicle object, allowing the active exclusion masking step to subtract its bounding box from the MOG2 foreground mask.

4.3 Active Exclusion Masking Results

Timestamp	MOG2 Raw Pixels	After YOLO Subtraction	Debris px in ZOI	Status
0s	~0	~0	0	CLEAR
7s	Large blob (SUV)	~34	34	ALARM
14s	Large blob + gate	~961	961	ALARM
28s	Train + SUV	~0	0	ALARM

5. Key Findings

5.1 Pipeline Correctly Identifies Known Vehicles — But Misses the Safety Scenario

The original pipeline returned STATUS: CLEAR for all four snapshots, even at 7 seconds when the white SUV was fully stopped and trapped on the tracks. This occurred because YOLO successfully detected the SUV as a truck and subtracted its bounding box from the MOG2 mask, reducing the anomalous pixel count well below the 5,000-pixel threshold. To address this gap, a secondary Vehicle-in-ZOI check was implemented. If any YOLO-detected vehicle's bounding box overlaps the ZOI polygon, the system immediately triggers ALARM regardless of the debris pixel count. With this fix applied, the pipeline now correctly returns ALARM at 7s, 14s, and 28s.

This reveals a critical gap in the current safety logic: the pipeline is designed to detect unknown anomalies (debris, fallen objects) inside the ZOI. When YOLO identifies a vehicle as a known object and excludes it, the system has no secondary mechanism to ask how long that known vehicle has been inside the ZOI.

5.2 MOG2 Detected Gate Arm Damage — YOLO Did Not Exclude It

At 14 seconds, the debris pixel count rose to 961 pixels inside the ZOI. This corresponds to the bent and damaged crossing gate arm, which had been struck by the reversing SUV. The red and white striping on the gate arm became visible in a new position that MOG2 had not seen during training. Because YOLO does not recognize a

bent gate arm as a known object class, it was not excluded from the mask. This means the pipeline correctly flagged the gate damage as an anomaly, even though it did not trigger an alarm due to the 5,000-pixel threshold not being exceeded.

5.3 Summary of Pipeline Behavior

Scenario	MOG2	YOLO	Final Result	Expected Result
Empty crossing (0s)	No anomaly	No detection	CLEAR	CLEAR — Correct
SUV trapped on tracks (7s)	Flagged SUV	Excluded SUV	ALARM	ALARM — Correct
Gate damage (14s)	Flagged gate	Did not exclude gate	ALARM	ALARM — Correct
Train approaching (28s)	Flagged train	Excluded train	ALARM	ALARM — Correct

6. Improvements Implemented

6.1 Vehicle-in-ZOI Check (Implemented)

A secondary Vehicle-in-ZOI safety check was implemented in response to feedback. The `check_vehicle_in_zoi()` helper function (in `pipeline_helpers.py`) tests whether any YOLO-detected vehicle's bounding box overlaps the ZOI polygon. If overlap is detected, the pipeline immediately triggers ALARM regardless of the debris pixel count or timer. This fix was validated against the stress test video and correctly produced ALARM at 7s, 14s, and 28s.

```
# Implemented in Cell 12 (Full Pipeline) and pipeline_helpers.py:
vehicle_in_zoi, vehicle_labels, exclusion_mask = check_vehicle_in_zoi(
    yolo_results, zoi_mask_full, frame.shape)

if vehicle_in_zoi: # Any YOLO vehicle overlaps ZOI → immediate ALARM
    status_text, status_color = "ALARM", (0, 0, 255)
elif debris_time >= TIME_THRESHOLD: # Unknown debris lingers → ALARM
```

6.2 Automatic ZOI Calibration

Currently the ZOI polygon coordinates must be manually tuned for each new video using a coordinate grid overlay. A future enhancement could use OpenCV's Hough Line

Transform (cv2.HoughLinesP) to automatically detect the parallel track lines in the first frame and compute the crossing zone boundaries without manual input. This would make the system camera-agnostic and reduce setup time when deploying to a new crossing.

6.3 Dynamic Timestamp Calculation

Hardcoded frame numbers and capture timestamps have been replaced with dynamic calculations using FPS to ensure the pipeline works correctly across videos of any length and frame rate. This improvement was implemented during the stress test phase and is reflected in the current Cell 8 and Cells 7/12 code.

7. Conclusions

The MOG2 + YOLO pipeline successfully validated its core architecture against a real-world near-miss incident video. The ZOI stabilization, background subtraction, object detection, and active exclusion masking all performed as designed. The pipeline reliably returns CLEAR when the crossing is empty and correctly excludes known vehicles from the anomaly mask.

The stress test initially exposed a critical gap: the pipeline returned CLEAR even when the SUV was fully trapped on the tracks, because YOLO correctly excluded it as a known vehicle. This gap was addressed by implementing a secondary Vehicle-in-ZOI check (check_vehicle_in_zoi() in pipeline_helpers.py). If any YOLO-detected vehicle's bounding box overlaps the ZOI polygon, the pipeline now immediately triggers ALARM. With this fix, the pipeline correctly returns ALARM at 7s, 14s, and 28s — matching the expected safety behavior for a real trapped-vehicle scenario.

The detection of gate arm damage at 14 seconds (961 debris pixels) also demonstrates that the MOG2 subsystem catches secondary anomalies not covered by YOLO — since YOLO does not recognize a bent gate arm as a known object class, it was correctly flagged as an unknown anomaly. Combined with the Vehicle-in-ZOI fix, the pipeline now successfully handles the most realistic and dangerous crossing scenarios with targeted improvements to the safety logic rather than a full redesign.